MATH4065 – Lab Session 1

Introduction

What are `R` and `Rstudio`?

`R` is a free version of a commercial statistical language/package called S-Plus. The commands in `R` and `S-Plus` are almost identical. There is a huge range of statistical commands in `R`, from basic summary statistics to cutting edge research applications. It is also a programming environment, so you can soon build up your own routines. Almost all practical statistical analysis is carried out on a computer, and so becoming familiar with computer packages and languages is an essential task for any statistician.

`R` can be downloaded from CRAN (Comprehensive R Archive Network):http://www.r-project.org (http://www.r-project.org). `R` is a command line based language, so you will need to type in the commands with any options and arguments. When `R` starts a command window is opened. It is here that you type in the commands. However, you are strongly encouraged to use `RStudio` which is a set of integrated tools designed to help you be more productive with `R`. It includes a console, syntax-highlighting editor that supports direct code execution, as well as tools for plotting, history, debugging and workspace management. `RStudio` is also free to use and can be downloaded from https://www.rstudio.com/ (https://www.rstudio.com/)

> In other words, `R` is the statistical software which performs computations, and `RStudio` is a nice graphical user interface, designed to make using R easier, by making it easier to type commands, get help, see graphs, load packages etc.

Starting `R/RStudio` on a University of Nottingham PC

R and RStudio are pre-installed on University PCs when you log in. To find RStudio (i) click on the Windows icon in the bottom left hand corner; (ii) scroll down the list and click on UoN Applications; (iii) scroll down until you see RStudio, then click on it to open.

How to get help

To find out how to use a function available to you, just type

```
?nameofthefunction
```

into the command line and the help file will open. Function help files almost always, after all the technical descriptions, also provide a useful example or two which you can copy and paste into R to see what it does. You can also type

```
example(functionname)
```

and it will work through the examples with you.

There is a HUGE number of books, tutorials, lecture notes on programming with `R` and other resources on the Web and you are encouraged to look at (some of) them. In addition, please speak to, and get help from, each other!

#Some basics ##Defining variables

As soon as you open `R`, then this brings you the `R` console.

The `>` symbol at the start is the `R` prompt, waiting for you to type a command. Type:

```
x<-1
```

This creates a variable called `x` in the memory and the `<-` arrow gives it the number 1. To find out what is stored in a variable, type the variable name into the console and press enter. For example, type:

```
x
```

```
## [1] 1
```

Try defining some more variables. Which of the following variable names are allowed?

1. `X`
2. `CH923`
3. `before after`
4. `4ever`
5. `sea.shore`

Note that variable names can have letters, numbers, full stops and underscores in them, but they can NOT have spaces or start with a number. They are also case-sensitive: foo, Foo and FOO are all different variables.

Try typing the following:

```
x<-x+1
```

What has happened to `x`? If this was an equation, $x = x + 1$, then it wouldn't make sense. However, `R` interprets the `x` on the right to be be the current value of `x`, adds 1 to it and the result is assigned as the new value of `x` on the left.

To list what variables you have in the memory then type

```
ls()
```

If you wish to remove variables from memory then use the `rm` command. For example,

```
rm(x)
```

removes the variable `x` from memory and

```
rm(list=ls())
```

deletes all the variables.

##Performing calculations

We can use `R` as a calculator. Using mental arithmetic, write down the answers to the following easy calculations on a piece of paper. Then use `R` to do the same sums.

1. `2*3` (Note: `*` means `multiply by'.)
2. `` `6-2+4/2 ``
3. `2*2^3` (Note: The hat symbol means `raise to the power of'.)

Did you get the answers you were expecting? If you didn't then it is because you didn't interpret the `operator precedence' in the same way as `R`. The order that `R` does things is

```
If you want to change the order, then you need to use brackets, so to calculate $3^{2\times2}
$, you would type

```r
3^(2*2)
```

```
## [1] 81
```

#Basic commands ##Vectors

Let us start by typing in a vector:

```
x<-c(1,5,3,7,6)
```

Here the `c()` means it's a column vector, and the `<-` assigns the vector of data $(1, 5, 3, 7, 6)$ into the variable named `x`. You can see what is in `x` by just typing

```
x
```

```
## [1] 1 5 3 7 6
```

We can see an individual element in `x` by using square brackets. Typing

```
x[2]
```

```
## [1] 5
```

will show you the second element of `x`.

Another way of defining vectors is using the colon. Define two more vectors,

```
y<-1:10
z<-1:3
```

and view them. Try to guess what will happen if you type the following commands into `R`. Which ones will cause a warning message?

1. `x+1`
2. `y*3`
3. `x+y[1:5]`
4. `x+z`
5. `y+10*z`

Note that you cannot add vectors of different lengths. Sometimes it is useful to check how many numbers are in the vector. Use the command

```
length(x)
```
```

```
## [1] 5
```

to confirm that there are 5 numbers in the vector `x`.

##Calling functions `R` contains many in-built functions, for example mathematical functions, functions for plotting, functions for producing random numbers and functions for programming.

To use a function, just type the name of the function and then round brackets, containing the `arguments` (the inputs to the function). For example try

```
sum(20,35)
```

```
## [1] 55
```

The most useful function is the `help()` function, which explains how to use any R function. For example, to find out about the sum function type

```
help(sum)
```

Functions can often take vectors as arguments. Define `y <- 1:10` as before and then type

```
sum(y)
```

```
## [1] 55
```

Functions can also have arguments that are optional. For example if you type

```
log(2)
```

```
## [1] 0.6931472
```

then `R` calculates the natural logarithm of 2. However, if you type

```
log(2,base=10)
```

```
## [1] 0.30103
```

then `R` gives a different answer - the base 10 logarithm of 2.

Usually it is possible to guess the name of the function that you need, but you can also search for functions by typing

```
?median
```

to search the help files for functions that calculate a median.

Let's now consider simulating 10000 observations from a standard normal $N(0,1)$ distribution. Type

```
x<-rnorm(10000)
```

and we can look at some summary statistics for the data using the following functions

```
summary(x)
mean(x)
sd(x)
var(x)
median(x)
```

Note that the `summary` command calculates the minimum, lower quartile, median, upper quartile and maximum as well as the mean. Note also that the `var` command calculates the sample variance, i.e. the unbiased estimate of the population variance, rather than the actual variance of `x`. We'll look at the `rnorm` command in more detail. Type

```
help(rnorm)
```

```
## starting httpd help server ... done
```

to obtain details of the command. You will notice the help lists details of some other commands that involve the normal distribution (`dnorm` , `pnorm` , `qnorm` , as well as `rnorm`), and it gives the full details of the arguments for the command `rnorm` in the form `rnorm(n, mean=0, sd=1)` .

If we want to simulate 10000 observations from a $N(2, 9)$ distribution then the command is

```
y <- rnorm(10000,2,3)
```

because we were specifying that the sample size is `n=10000` , the sample mean is `mean=2` and the sample standard deviation is `sd=3` . We could have used the long-winded version of the command

```
rnorm(n=10000,mean=2,sd=3)
```

which is the same as `rnorm(10000,2,3)` .

When we issued the first simulation command `rnorm(10000)` we didn't specify what the mean and standard deviation were, so `R` uses the default values of 0 and 1, which can be found using `help(rnorm)` .

Useful hint

If you press the up arrow then you get the last thing that you typed in. If you press it again you get the previous command to that. This is particularly useful if you had a small typo in a command or are carrying out lots of similar commands. Tab is also useful as it auto-completes commands.

#Simple graphics ##Scatterplots

We might want to plot our simulated values in y versus those in x. The command is

```
plot(x,y)
```

Note the `x` (horizontal axis) comes first before the `y` (vertical axis). There are numerous options for the plots that can be used, e.g. specifying the axes limits

```
plot(x,y,xlim=c(-4,4),ylim=c(-15,15))
```

We can also add a title `main` , and label the x and y axes.

```
plot(x,y,xlim=c(-4,4),ylim=c(-15,15),main="A simple plot",xlab="x",ylab="y")
```

Let us add the point $(0, 1)$ to the plot, which is coloured red (col=2), and given a different plotting symbol, a triangle, (pch = 2)

```
points(0,1,col=2,pch=2)
```

All the different graphical parameters are found by looking at `help(par)`, and there are a lot of them!

##Histograms

We might want to plot a histogram of the values in x. The command is

```
hist(x)
```

We can change the number of bars in the histogram by suggesting an alternative number of breaks. For example,

```
hist(x,breaks=100)
```

creates a histogram with approximately 100 bars. Alternatively, we can specify exactly where the breaks between columns should be with a vector such as

```
hist(x,breaks=c(-6,-4,-2,-1,0,1,2,4,6))
```

By default, `R` will have the vertical axis as frequency, i.e. the number of observations in each class. We can change the vertical axis to density using

```
hist(x,freq=FALSE)
```

This can be useful when assessing if the variable follows a particular distribution as the area under the histogram is now 1, just as it should be for a probability density function.

Note that you can add or change the title and axis labels using the `main`, `xlab` and `ylab` commands as before.

##Boxplots

A simple boxplot can be plotted using

```
boxplot(x,range=0)
```

##Line graphs

Now let's plot a line graph of $y = x^2$. To do this, we need to evaluate y at several values of x, and change the type of graph. Type

```
x<--10:10
y<-x^2
plot(x,y,type="l")
```

Note that the `l` is an abbreviation for lines. We can add another line, $\frac{1}{2}x^2$ in blue, to an existing plot using

```
lines(x,0.5*x^2,col="blue")
```

##Other commands

Some other useful plotting commands are, `pie` (for pie charts), `qqnorm` (for QQ plots), `legend` (to add a legend to a plot) and `text` (to add text to a plot).

##Printing Graphs It's great to have plots on the screen, but you will often need to get them on paper to include in reports etc. `R` treats the screen as one device and a postscript/pdf file (which you can print) as another. You can save a graph you have on the screen as a postscript file like this:

```
dev.copy2eps(file="foo.eps")
```

The resulting file `foo.eps` can be printed, viewed on screen or included in a report. Note that you can specify which directory the file is saved in by clicking on *File* and *Change dir*. If you want `R` to generate a postscript file without displaying anything on the screen, you can do it by using `postscript()` before your plotting commands and `dev.off()` after them. Here is how someone can do this

```
postscript(file="foo.eps", onefile = FALSE,horizontal=FALSE,width=6,height=5)
plot(x,y,type="l")
dev.off()
```

If you want to save your file as `.pdf` file then you should type something like

```
dev.print(pdf, file="foo.pdf")
```

Alternatively, if you are using R-Studio then you can hit the button `Export` and the whole procedure of saving the image into file becomes easier.

Exercises

Try the following:

Exercise 1.

Produce a line graph of the probability density function of a normal distribution with mean 5 and standard deviation 2. Note that you can use the `dnorm` function to calculate the pdf values. Try to make the line smooth by evaluating the function at lots of points - you may find the `seq` function helpful.

Exercise 2.

Simulate 50 random observations from a normal distribution with mean 5 and standard deviation 2. Plot two histograms of your simulated values - one with frequency on the vertical axis and one with density.

Exercise 3.

Use the `lines` function to combine your answers from the first two questions on the same plot. The histogram and the pdf should have approximately the same shape. What happens if you change the number of random observations in the histogram?

Exercise 4.

Simulate 500 random observations from a normal distribution with mean 0 and standard deviation 1. Plot a QQ plot of your simulated values and then use the command `abline(0,1)` to draw a line with y-intercept 0 and gradient 1 over the top. What do you notice? Try this again but changing the mean and standard deviation. What changes?

#Reading in data

Many people find reading data into R can be a challenge at first. Consider the Scottish hill race data in the MASS library

```
library(MASS)
```

which contains three columns of data on 35 different hill races in Scotland. It gives the distance, height of the climb, and the record time for each race. To look at the data type

```
hills
```

and you will see it has columns labelled `dist`, `climb` and `time`, and rows labelled by the name of the race. A dataframe is just a matrix with some names labelling the columns. We can change the names if we want

```
names(hills)<-c("distance", "height.climbed", "record")
```

and then see what the data frame looks like

```
hills
```

We can look at a matrix of pairwise scatterplots:

```
pairs(hills)
```

Or make the columns available by name

```
attach(hills)
```

so that we can now just refer to

```
distance
```

for example. We can now plot time against distance using

```
plot(distance, record)
```

We can identify outlying points by typing

```
identify(distance, record, row.names(hills))
```

using the left mouse button to click on points to see the name of the race, and the right button to quit.

Typically, the data will be in a file and you will want `R` to read it in and then make a plot of it. As an example, we will be using the following dataset about cars in the US which consists of 93 rows and 7 columns. You can download the data from the module moodle page.

Save the above file as `Cars.csv` in your directory. Click on *File* and *Change dir* and select the directory where you saved the file. The command `read.csv` is used to read the file into `R`:

```
cars.data <- read.csv("Cars.csv", header=TRUE)
```

the argument `header=TRUE` is there to let `R` know that the first row contains the labels of the columns, which are:

- `Model`: Model of the car.

- `Type` : Type: a factor with levels “Small”, “Sporty”, “Compact”, “Midsize”, “Large” and “Van”.
- `Min.Price` : Minimum Price (in $1,000): price for a basic version.
- `Price` : Midrange Price (in $1,000): average of Min.Price and Max.Price.
- `Max.Price` : Maximum Price (in $1,000): price for a premium version.
- `MPG.city` : City MPG (miles per US gallon by EPA rating).
- `MPG.highway` : Highway MPG.

Entering the command

```
attach(cars.data)
```

allows us to refer to the columns by their names. Try the following commands and see what happens.

```
table(Type)
boxplot(Price~Type,range=0)
```

Suppose we are only interested in the cars which can do at least 30 miles per gallon in the city. We can use the following commands to find these and calculate some useful statistics.

```
MPG.city>=30
Price[MPG.city>=30]
summary(Price[MPG.city>=30])
```

The first command returns a vector saying whether or not the variable `MPG.city` is greater than or equal to 30. The second command returns the prices of cars for which the first command was true and the final command calculates the usual summary statistics for `Price` but this time only using the subset of cars which met our condition. If we are working with categorical variables then we can use a condition such as

```
Type=="Compact"
```

##Exercises##

Try the following questions.

Exercise 1.

Calculate the mean and standard deviation of city MPG for vehicles which cost more than $\$30,000$.

Exercise 2.

Produce a scatter plot of price against city MPG. What relationship do you notice?

Exercise 3.

Which vehicle has the largest price difference between its premium and base versions?

Exercise 4.

Produce a histogram of highway MPG with an appropriate number of columns.

Exercise 5.

Produce a boxplot of highway MPG for each type of vehicle on the same graph. Make sure you give your plot an appropriate title and label each axis.

#Matrices and Lists ##Matrices

`R` allows you to arrange your data into a matrix (which is a 2-dimensional array!) or indeed an array with any number of dimensions. An array in `R` is a vector with a set of dimensions attached. Consider the vector

```
a.vector <- c(3, 5, 6, 2.5, 100, 27.7)
```

We can make a copy of our vector and convert it into a 2 (rows) by 3 (column) array, like this:

```
a.matrix<-matrix(a.vector,2,3)
a.matrix
```

```
##      [,1] [,2]  [,3]
## [1,]    3  6.0 100.0
## [2,]    5  2.5  27.7
```

Note that the elements of `a.vector` are inserted in the matrix column by column. If we want to access parts of the matrix, we do it like this:

```
a.matrix[1,2]
```

```
## [1] 6
```

```
a.matrix[1,]
```

```
## [1]   3   6 100
```

```
a.matrix[,2]
```

```
## [1] 6.0 2.5
```

```
a.matrix[1,2:3]
```

```
## [1]   6 100
```

Note `a.matrix[i,j]` refers to the ith row and the jth column - the order is important. Not specifying a row or column returns the entire column or row.

It is easy to perform arithmetic operations on matrices in `R`. For example, we can add a constant to all the elements in the matrix by

```
x<-matrix(c(2,4,6,8),nrow=2)
x+10
```

```
##      [,1] [,2]
## [1,]   12   16
## [2,]   14   18
```

or we can multiply each element by itself:

```
x*x
```

```
##      [,1] [,2]
## [1,]    4   36
## [2,]   16   64
```

Note that the `*` sign multiplies element-by-element. To do a matrix multiplication, you need the special operator `%*%` :

```
x%*%x
```

```
##      [,1] [,2]
## [1,]   28   60
## [2,]   40   88
```

`R` is a very convenient environment for working with matrices. Why? This is because:

- a matrix is displayed the way round you expect;
- the indices are the same way round as in mathematical texts
- there are built-in functions to find the inverses, determinants and eigenvectors, to solve sets of simultaneous equations and so on.

Let us look at some more matrix based commands. Load the Fisher's Iris data, which is a standard dataset in R, into `x` .

```
data(iris)
x <- iris
```

Then,

```
y <- x[, 1:4]
```

extracts the first four columns from the dataset. We can convert a dataframe into a matrix with

```
z<-as.matrix(y)
```

We can transpose a matrix using the `t` function and check the dimensions of a matrix using the `dim` function.

```
A<-t(z)%*%z
dim(A)
```

```
## [1] 4 4
```

We can find the eigenvalues and eigenvectors of the symmetric matrix A as follows

```
E<-eigen(A,symmetric=TRUE)
```

##Lists Note that the output of the previous command is a list, which has two parts. For some purposes, arrays are not flexible enough. One might want a variable to contain elements of different types. There is `E\$values` (the eigenvalues) and `E\$vectors` (the eigenvectors). We can create a list as follows:

```
ans <- list(x=0,y=0,z="")
```

where the list contains `$x` and `$y` which are real numbers and `$z` which is a character string. So, we can make an assignment

```
ans$x <- c(1,2,3,4,5)
ans$y <- c(4,3,2,1)
ans$z <- c("cat","dog")
```

and the list can be seen by typing

```
ans
```

```
## $x
## [1] 1 2 3 4 5
##
## $y
## [1] 4 3 2 1
##
## $z
## [1] "cat" "dog"
```

We can refer to list elements with commands such as,

```
ans$x
```

```
## [1] 1 2 3 4 5
```

```
ans[[2]]
```

```
## [1] 4 3 2 1
```

The name of the list object followed by a `[[i]]` returns us the ith element of that list. Therefore, the order in which the objects are stored in is important. Lists can be very useful in providing the output of functions.

##Exercises

Exercise 1.

Find the solution to this set of simultaneous equations:

$$\begin{aligned} 2x_1 + x_2 &= 1 \\ x_1 + 3x_2 + x_3 &= -3 \\ x_2 + 2x_3 &= 1 \end{aligned}$$

Exercise 2.

Find the eigenvalues and eigenvectors of this matrix:

$$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$